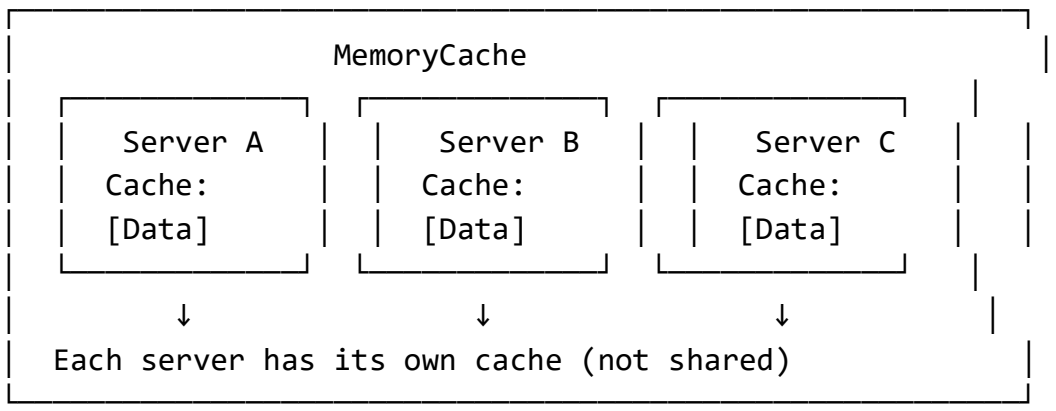
MemoryCache vs Redis - Complete Guide with Interview Preparation

Part 1: MemoryCache - Why Used & How to Implement

What is MemoryCache?

MemoryCache is like your computer's RAM for storing temporary data. It's fast but only exists while your application is running.



Why Use MemoryCache?

Benefits:

- **Super Fast**: In-memory access (nanoseconds)
- **Simple**: No external dependencies
- **Free**: Built into .NET

Limitations:

- **Local Only**: Each server has its own cache
- **Lost on Restart**: Cache disappears when app restarts
- **Memory Limited**: Limited by server RAM

How to Implement MemoryCache

Step 1: Add NuGet Package

```
dotnet add package Microsoft.Extensions.Caching.Memory
```

Step 2: Configure in Program.cs

```
// Add MemoryCache service
builder.Services.AddMemoryCache();
```

Step 3: Use in Your Code

Example: Catalog Service Product Caching

```
using Microsoft.Extensions.Caching.Memory;

public class ProductRepository : IProductRepository
{
    private readonly IMemoryCache _memoryCache;
    private readonly IProductRepository _productRepository; // Your
actual repository

    public ProductRepository(IMemoryCache memoryCache,
IProductRepository productRepository)
    {
        _memoryCache = memoryCache;
        _productRepository = productRepository;
    }

    public async Task<Product> GetProduct(string productId)
    {
        // Try to get from cache first
        if (_memoryCache.TryGetValue($"product_{productId}", out
Product cachedProduct))
        {
            return cachedProduct;
```

```
        }

        // Not in cache, get from database
        var product = await
_productRepository.GetProductFromDatabase(productId);

        if (product != null)
        {
            // Store in cache for 5 minutes
            var cacheOptions = new MemoryCacheEntryOptions
            {
                AbsoluteExpirationRelativeToNow =
TimeSpan.FromMinutes(5),
                SlidingExpiration = TimeSpan.FromMinutes(2)
            };

            _memoryCache.Set($"product_{productId}", product,
cacheOptions);
        }

        return product;
    }
}
```

Step 4: Cache Options Explained

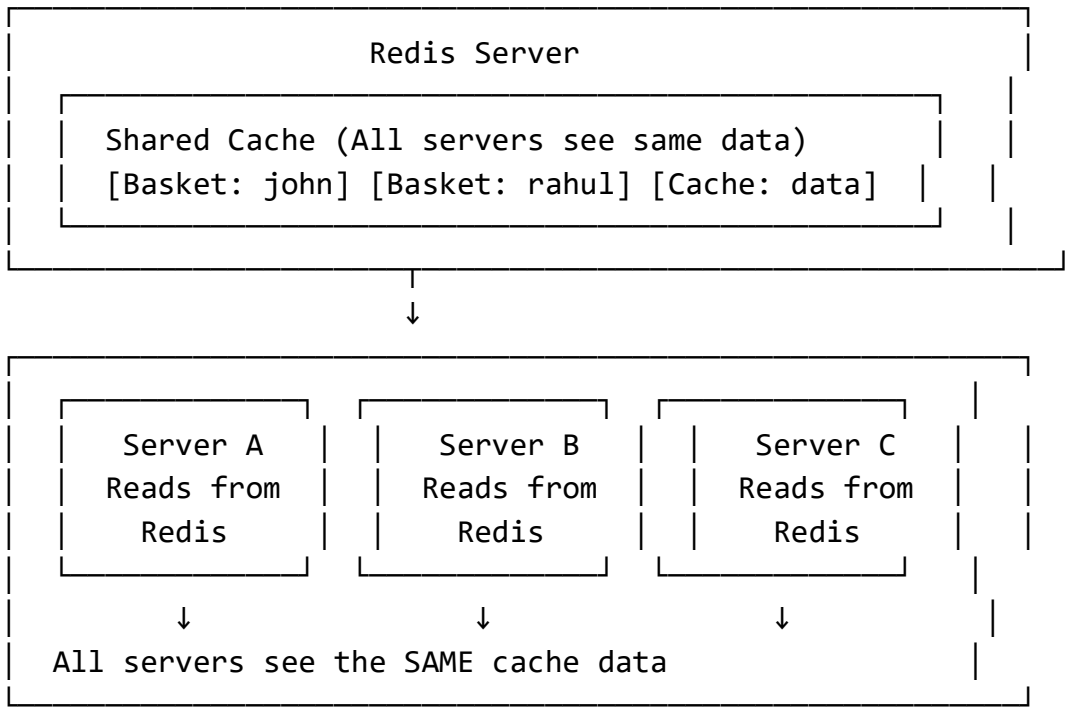
```
var cacheOptions = new MemoryCacheEntryOptions
{
    // Cache expires after 5 minutes (absolute)
    AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5),

    // Cache expires if not accessed for 2 minutes (sliding)
    SlidingExpiration = TimeSpan.FromMinutes(2),

    // Set priority for cache eviction
    Priority = CacheItemPriority.Normal,

    // Size limit (1MB)
```

```
    Size = 1024 * 1024
};
```

Part 2: Redis - Why Used & How to Implement

What is Redis?

Redis is a distributed in-memory cache that multiple servers can share. It's like a shared notebook that all your services can read from and write to.



Why Use Redis?

Benefits:

- **Distributed**: Multiple servers share the same cache
- **Persistent**: Cache survives app restarts
- **Scalable**: Can handle millions of operations per second
- **Data Structures**: Lists, sets, sorted sets, hashes

When to Use Redis:

- **Shopping carts** (like in your Basket service)
- **User sessions** across multiple servers
- **Rate limiting**
- **Real-time leaderboards**
- **Caching expensive database queries**

How Redis Solves Problems in Your Project

Your Basket Service Problem:

```
User adds items to cart on Server A
    ↓
User refreshes on Server B
    ↓
Server B doesn't know about cart from Server A!
```

Redis Solution:

```
User adds items to cart → Store in Redis
    ↓
User refreshes on any server → Read from Redis
    ↓
All servers see the same cart!
```

How You Implemented Redis in Your Project

Step 1: Add NuGet Packages

File: Basket.Infrastructure/Basket.Infrastructure.csproj

```
<PackageReference
Include="Microsoft.Extensions.Caching.StackExchangeRedis"
Version="10.0.1" />
<PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
```

Step 2: Configure Redis in Program.cs

File: Basket.API/Program.cs (Lines 66-70)

```
// Redis Configuration
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration =
builder.Configuration.GetSection("CacheSettings")
                                                    .GetValue<string>("Co
nnectionString");
});
```

Step 3: Connection String in appsettings.json

File: Basket.API/appsettings.json

```
{
  "CacheSettings": {
    "ConnectionString": "localhost:6379"
  }
}
```

Step 4: Create Repository

File: Basket.Infrastructure/Repositories/BasketRepository.cs

```
using Microsoft.Extensions.Caching.Distributed;
using Newtonsoft.Json;

public class BasketRepository : IBasketRepository
{
    private readonly IDistributedCache _redisCache;

    public BasketRepository(IDistributedCache redisCache)
    {
        _redisCache = redisCache;
    }
```

```
    public async Task<ShoppingCart> GetBasket(string userName)
    {
        var basket = await _redisCache.GetStringAsync(userName);
        if (string.IsNullOrEmpty(basket))
        {
            return null;
        }
        return JsonConvert.DeserializeObject<ShoppingCart>(basket);
    }

    public async Task<ShoppingCart> UpsertBasket(ShoppingCart
shoppingCart)
    {
        // Convert C# object to JSON and store in Redis
        await _redisCache.SetStringAsync(
            shoppingCart.UserName,
            JsonConvert.SerializeObject(shoppingCart)
        );
        return await GetBasket(shoppingCart.UserName);
    }

    public async Task DeleteBasket(string userName)
    {
        await _redisCache.RemoveAsync(userName);
    }
}
```

Step 5: Use in Controller

File: Basket.API/Controllers/BasketController.cs

```
[HttpGet("{userName}")]
public async Task<ActionResult<ShoppingCart>> GetBasket(string
userName)
{
    var basket = await _basketRepository.GetBasket(userName);
    return Ok(basket);
```

```
}

[HttpPost]
public async Task<ActionResult<ShoppingCart>> UpdateBasket([FromBody]
ShoppingCart basket)
{
    var updatedBasket = await _basketRepository.UpsertBasket(basket);
    return Ok(updatedBasket);
}
```

Part 3: MemoryCache vs Redis - Comparison with Your Project

Comparison Table

Feature	MemoryCache	Redis
Speed	Faster (nanoseconds)	Fast (microseconds)
Scope	Local to one server	Distributed (shared across servers)
Persistence	Lost on restart	Survives restarts
Scalability	Limited by server RAM	Can scale horizontally
Dependencies	Built into .NET	Requires Redis server
Your Usage	Not implemented	✅ Basket service

When to Use Which

Use MemoryCache For:

- **Static reference data** (product categories, countries)
- **Frequently accessed, rarely changing data**
- **Single-server applications**
- **Temporary computations**

Example: Product categories in Catalog service

```
// Good for MemoryCache
public async Task<IEnumerable<ProductCategory>> GetCategories()
{
    return await _memoryCache.GetOrCreateAsync("categories",
        TimeSpan.FromHours(1),
        () => _database.GetCategories());
}
```

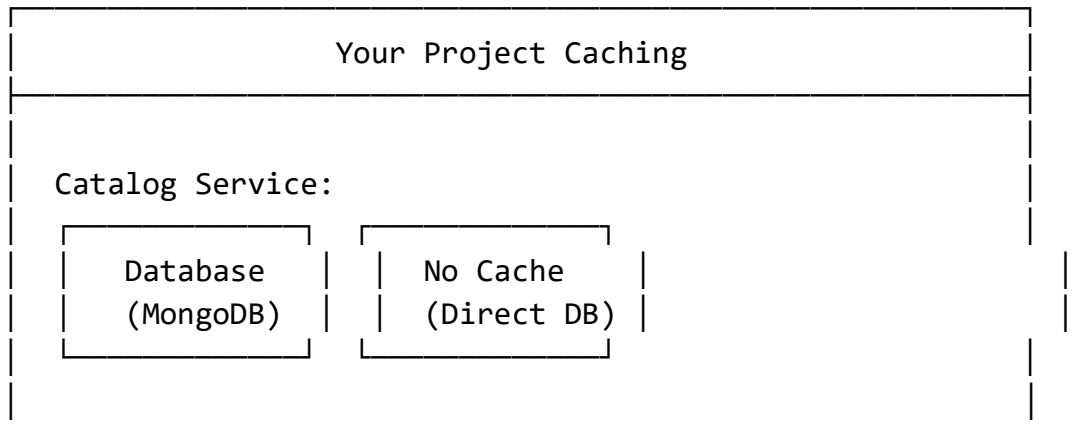
Use Redis For:

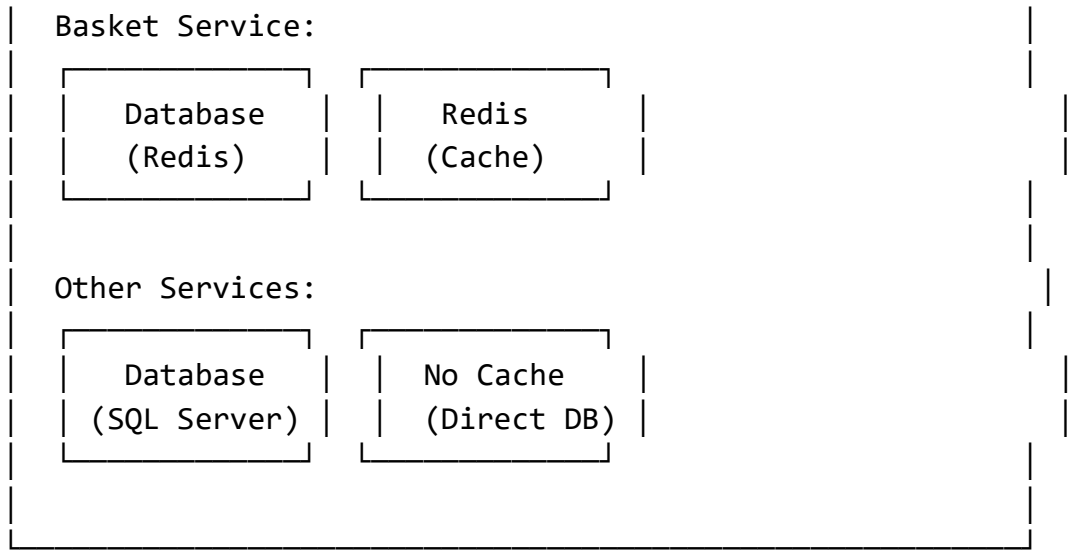
- **User-specific data** (shopping carts, sessions)
- **Data that must be shared across servers**
- **Data that must survive restarts**
- **Real-time features**

Example: Shopping cart in Basket service (your implementation)

```
// Good for Redis (shared across servers)
public async Task<ShoppingCart> GetBasket(string userName)
{
    var basket = await _redisCache.GetStringAsync(userName);
    return JsonConvert.DeserializeObject<ShoppingCart>(basket);
}
```

Your Project's Current State





Recommendation for Your Project

Add MemoryCache to Catalog Service:

```
// In Catalog.API/Program.cs
builder.Services.AddMemoryCache();

// In Catalog.Infrastructure/Repositories/ProductRepository.cs
public class ProductRepository : IProductRepository
{
    private readonly IMemoryCache _memoryCache;
    private readonly IMongoCollection<Product> _products;

    public async Task<Product> GetProduct(string productId)
    {
        return await
_memoryCache.GetOrCreateAsync($"product_{productId}",
            TimeSpan.FromMinutes(10),
            () => _products.Find(p => p.Id ==
productId).FirstOrDefaultAsync());
    }
}
```

Part 4: Interview Preparation Q&A

Basic Questions

Q1: What is the difference between MemoryCache and Redis?

A:

- **MemoryCache**: In-memory cache, local to one server, lost on restart, faster (nanoseconds)
- **Redis**: Distributed cache, shared across servers, persistent, slightly slower (microseconds), requires external server

Q2: When would you use MemoryCache vs Redis?

A:

- **MemoryCache**: Static reference data, single-server apps, temporary computations
- **Redis**: User sessions, shopping carts, data shared across servers, data that must survive restarts

Q3: How do you implement Redis in .NET?

A:

1. Add `Microsoft.Extensions.Caching.StackExchangeRedis` package
2. Configure with `builder.Services.AddStackExchangeRedisCache()`
3. Inject `IDistributedCache` in constructor
4. Use `SetStringAsync()`, `GetStringAsync()`, `RemoveAsync()`

Intermediate Questions

Q4: Why did you choose Redis for your Basket service instead of MemoryCache?

A: Because shopping carts need to be:

- **Shared across multiple server instances**
- **Persistent** (survive application restarts)
- **Accessible from different servers** (load balancer scenarios)
- **Consistent** for the same user regardless of which server handles the request

Q5: How does Redis solve the distributed caching problem in microservices?

A: Redis provides a centralized cache that all microservices can access:

- Basket service stores cart data in Redis
- Any instance of Basket service can read the same cart
- Data survives service restarts and deployments
- Enables horizontal scaling without losing user data

Q6: What happens if Redis goes down?

A:

- Application should handle exceptions gracefully
- Can implement fallback to database
- Use retry policies with exponential backoff
- Consider implementing a local cache fallback strategy

Advanced Questions

Q7: How do you handle cache invalidation in Redis?

A:

- **Time-based expiration**: Set TTL on cache keys
- **Event-driven**: Publish cache invalidation events via RabbitMQ
- **Manual invalidation**: Remove keys when data changes
- **Cache tags**: Use Redis sets to manage related cache keys

Q8: How would you implement a cache-aside pattern with Redis?

A:

```
public async Task<Product> GetProduct(string productId)
{
    // Try cache first
    var cached = await
_redisCache.GetStringAsync($"product_{productId}");
    if (cached != null)
        return JsonConvert.DeserializeObject<Product>(cached);
```

```
    // Cache miss - get from database
    var product = await _database.GetProduct(productId);

    // Store in cache with expiration
    await _redisCache.SetStringAsync($"product_{productId}",
        JsonConvert.SerializeObject(product),
        TimeSpan.FromMinutes(10));

    return product;
}
```

Q9: What are the performance implications of JSON serialization in Redis?

A:

- **Pros**: Human-readable, easy to debug
- **Cons**: Slower than binary formats, larger size
- **Alternatives**: Use MessagePack, Protobuf for better performance
- **Trade-off**: For most cases, JSON is fine; optimize only when needed

Scenario Questions

Q10: Your Basket service is getting slow due to Redis calls. How would you optimize it?

A:

1. **Add local MemoryCache** as L1 cache (Redis as L2)
2. **Use binary serialization** (MessagePack instead of JSON)
3. **Batch Redis operations** (pipeline multiple commands)
4. **Implement cache warming** for frequently accessed carts
5. **Use Redis clustering** for better performance
6. **Monitor Redis metrics** to identify bottlenecks

Q11: How would you implement cache consistency across multiple services?

A:

- **Event-driven invalidation**: Publish events when data changes

- **Versioned cache keys**: Include data version in cache key
- **Distributed transactions**: Use Redis transactions
- **Read-through/write-through patterns**: Centralize cache logic

Your Project-Specific Questions

Q12: In your ECommerce project, why does only the Basket service use Redis?

A:

- **Basket service**: Needs shared cache for shopping carts across multiple instances
- **Other services**: Could benefit from MemoryCache for static data
- **Design decision**: Shopping carts are user-specific and need persistence
- **Architecture**: Each service caches data appropriate to its domain

Q13: How would you add caching to your Catalog service?

A:

```
// Add to Program.cs
builder.Services.AddMemoryCache();

// In ProductRepository
public async Task<Product> GetProduct(string productId)
{
    return await _memoryCache.GetOrCreateAsync($"product_{productId}",
        TimeSpan.FromMinutes(10),
        () => _products.Find(p => p.Id ==
productId).FirstOrDefaultAsync());
}
```

Q14: What caching strategies are you NOT using in your project?

A:

- **Write-behind cache**: Async database writes
- **Read-through cache**: Automatic cache population
- **Cache warming**: Pre-populating cache
- **Multi-level caching**: Local + distributed cache

- **Cache partitioning**: Distributing cache load

Practical Implementation Questions

Q15: Show me how you'd implement a distributed cache lock with Redis

A:

```
public async Task<bool> TryLockAsync(string key, TimeSpan expiration)
{
    var lockKey = $"lock:{key}";
    var lockValue = Guid.NewGuid().ToString();
    var success = await _redisCache.SetStringAsync(lockKey, lockValue,
expiration);
    return success;
}

public async Task<bool> ReleaseLockAsync(string key, string lockValue)
{
    var lockKey = $"lock:{key}";
    var script = "if redis.call('GET', KEYS[1]) == ARGV[1] then return
redis.call('DEL', KEYS[1]) else return 0 end";
    var result = await _redisCache.ScriptEvaluateAsync(script,
lockKey, lockValue);
    return (int)result == 1;
}
```

Questions You Might Be Asked

Q16: What's the difference between IDistributedCache and IOptions?

A:

- **IDistributedCache**: For caching data (Redis, MemoryCache)
- **IOptions**: For configuration settings (appsettings.json)
- **Different purposes**: One for runtime data, one for startup configuration

Q17: How do you monitor cache performance?

A:

- **Redis**: Use Redis CLI commands (INFO, MONITOR)
- **Application**: Add logging for cache hits/misses
- **Metrics**: Track cache hit ratio, response times
- **Alerting**: Set up alerts for cache failures

Q18: What would happen if you used MemoryCache instead of Redis in your Basket service?

A:

- **Problem**: Each server instance would have its own cache
- **User Experience**: User adds items on Server A, refreshes on Server B → empty cart
- **Data Loss**: Cache lost on every deployment/restart
- **Scalability Issues**: Cannot horizontally scale effectively

Final Tips

Key Points to Remember:

- **Redis** = Distributed, shared, persistent
- **MemoryCache** = Local, fast, temporary
- **Use both**: MemoryCache for static data, Redis for shared data
- **Your project**: Only uses Redis for baskets (good architecture)
- **Cache invalidation**: Time-based, event-driven, or manual
- **Performance**: Monitor hit ratio, optimize when needed

Common Mistakes to Avoid:

- Using MemoryCache for user-specific data
- Not handling cache failures gracefully
- Forgetting to set expiration times
- Caching too much data (memory pressure)
- Not monitoring cache performance

Summary

I've provided a complete guide covering:

1. MemoryCache

- **Why used**: Fast, in-memory, local caching
- **Implementation**: `AddMemoryCache()`, `IMemoryCache`, cache options
- **Best for**: Static data, single-server apps, temporary computations

2. Redis

- **Why used**: Distributed, shared across servers, persistent
- **Problem solved**: Shopping cart consistency across multiple instances
- **Implementation**: `AddStackExchangeRedisCache()`, `IDistributedCache`, JSON serialization

3. Comparison

- **MemoryCache**: Faster but local, lost on restart
- **Redis**: Slightly slower but distributed, persistent
- **Your project**: Only uses Redis for baskets (correct choice)

4. Interview Preparation

- **18+ questions** from basic to advanced
- **Real code examples** from your project
- **Scenario questions** with practical answers
- **Common mistakes** to avoid

Key Takeaways for Interviews

- **Redis for**: User sessions, shopping carts, data shared across servers
- **MemoryCache for**: Static reference data, single-server applications
- **Your architecture**: Good choice using Redis for baskets
- **Improvement**: Add MemoryCache to Catalog service for product data

This comprehensive guide should prepare you for any caching-related interview questions and help you explain your project's caching strategy effectively.